



Logic Design

Ch 5

Combinational Circuits

Instructor : Hasan Al Serhan

IT Department

Al Balqa Applied University



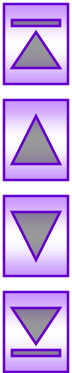
Combinational Circuits: Design Methods/Arithmetic Circuits

- Introduction
- Analysis Procedure
- Design Methods
- Gate-level (SSI) Design
 - ❖ Half Adder
 - ❖ Full Adder
 - ❖ BCD-to-Excess-3 Code Converter
- Block-Level Design
 - ❖ 4-bit Parallel Adder
 - ❖ BCD-to-Excess-3 Code Converter
 - ❖ 16-bit Parallel Adder
 - ❖ 4-bit Parallel Adder cum Subtractor



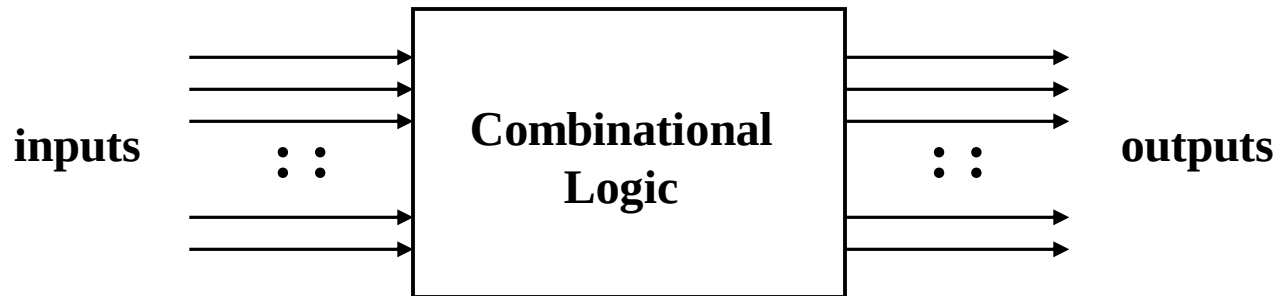
Combinational Circuits: Design Methods/Arithmetic Circuits

- Arithmetic Circuits
 - ❖ Adders
 - ❖ Parallel Adders
 - ❖ Cascading Adders
 - ❖ Parallel Adder-Subtractor
 - ❖ Comparator

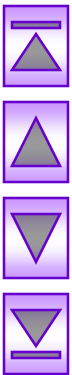


Introduction

- Two classes of logic circuits:
 - ❖ combinational
 - ❖ sequential
- **Combinational Circuit:**

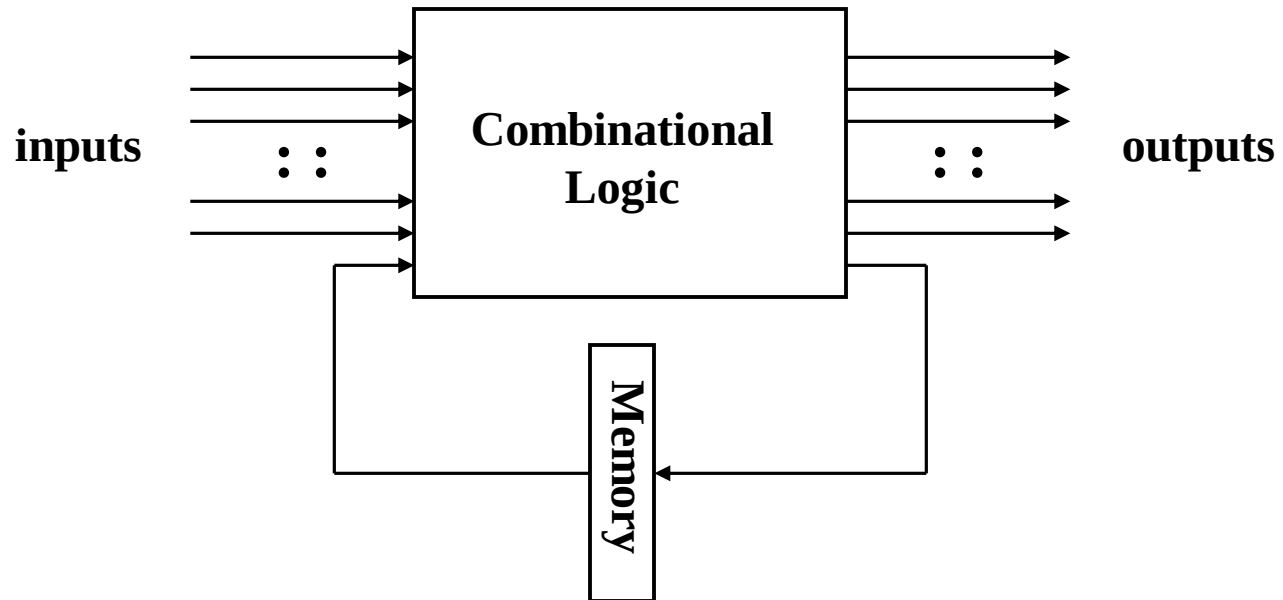


Each output depends entirely on the immediate (present) inputs.

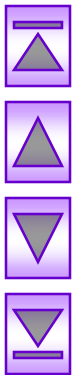


Introduction

- **Sequential Circuit:** (to be covered later)

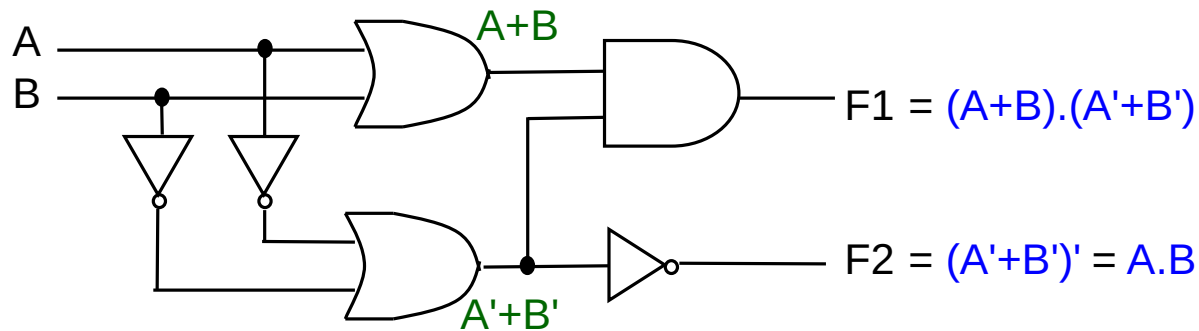


Output depends on both *present* and *past* inputs.
Memory (via feedback loop) contains past information.



Analysis Procedure

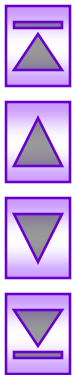
- Given a combinational circuit, can you analyze its function?



- Steps:

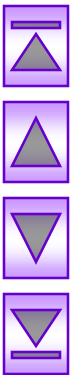
- ❖ 1. Label the inputs and outputs.
- ❖ 2. Obtain the functions of intermediate points and the outputs.
- ❖ 3. Draw the truth table.
- ❖ 4. Deduce the functionality of the circuit ➡ half adder.

A	B	(A+B)	(A'+B')	F1	F2
0	0	0	1	0	0
0	1	1	1	1	0
1	0	1	1	1	0
1	1	1	0	0	1



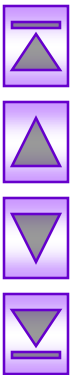
Design Methods

- Different combinational circuit design methods:
 - ❖ Gate-level method (with logic gates)
 - ❖ Block-level design method
- Design methods make use of logic gates and useful functional blocks.
 - ❖ These are available as Integrated Circuit (IC) chips.



Design Methods

- Type of IC chips (based on packing density) :
 - ❖ Small-scale integration (SSI): up to 12 gates
 - ❖ Medium-scale integration (MSI): 12-99 gates
 - ❖ Large-scale integration (LSI): 100-9999 gates
 - ❖ Very large-scale integration (VLSI): 10,000-99,999 gates
 - ❖ Ultra large-scale integration (ULSI): > 100,000 gates
- Main objectives of circuit design:
 - ❖ (i) reduce cost
 - reduce number of gates (for SSI circuits)
 - reduce IC packages (for complex circuits)
 - ❖ (ii) increase speed
 - ❖ (iii) design simplicity (reuse blocks where possible)



Gate-level (SSI) Design: Half Adder

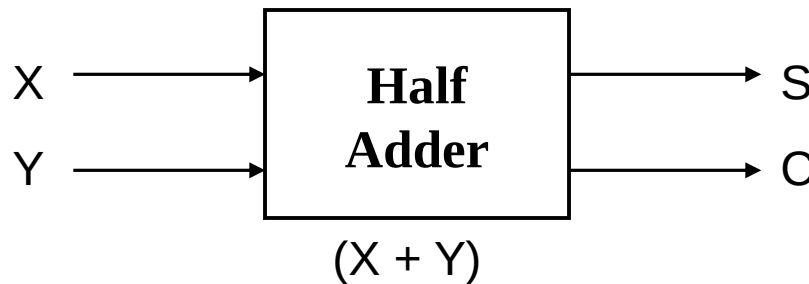
- Design procedure:

- 1) State Problem

Example: Build a **Half Adder** to add two bits

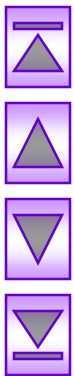
- 2) Determine and label the inputs & outputs of circuit.

Example: Two inputs and two outputs labelled, as follows:



X	Y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

- 3) Draw truth table.



Gate-level (SSI) Design: Half Adder

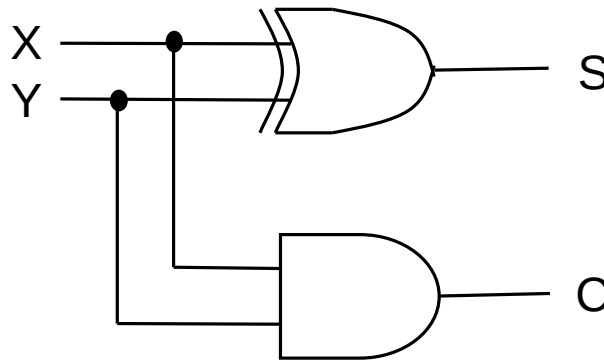
4) Obtain simplified Boolean function.

Example: $C = XY$

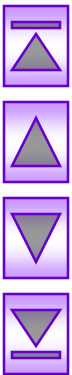
$$S = X'Y + XY' = X \oplus Y$$

X	Y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

5) Draw logic diagram.



Half Adder



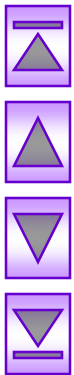
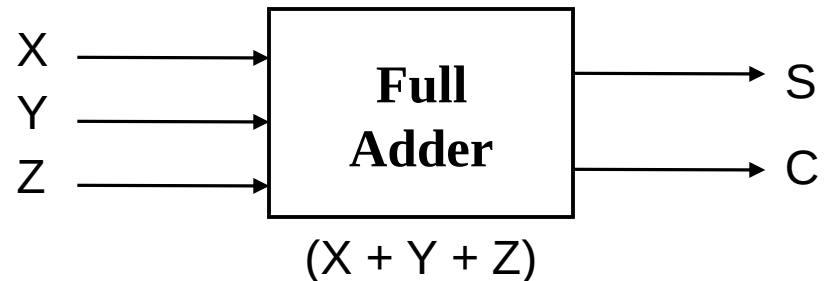
Gate-level (SSI) Design: Full Adder

- Half-adder adds up only two bits.
- To add two binary numbers, we need to add 3 bits (including the *carry*).

■ Example:

		1	1	1		carry
		0	0	1	1	X
+		0	1	1	1	Y
		1	0	1	0	S

- Need **Full Adder** (so called as it can be made from two half-adders).



Gate-level (SSI) Design: Full Adder

- Truth table:

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Note:

Z - carry in (to the current position)

C - carry out (to the next position)

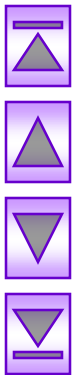
		C			
		YZ			
		00	01	11	10
X	0			1	
	1		1	1	1

- Using K-map, simplified SOP form:

$$C = XY + XZ + YZ$$

$$S = X'Y'Z + X'YZ' + XY'Z' + XYZ$$

		S			
		YZ			
		00	01	11	10
X	0		1		1
	1	1		1	

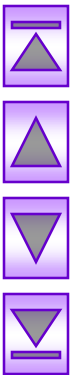


Gate-level (SSI) Design: Full Adder

- Alternative formulae using algebraic manipulation:

$$\begin{aligned}C &= XY + XZ + YZ \\&= XY + (X + Y)Z \\&= XY + ((X \oplus Y) + XY)Z \\&= XY + (X \oplus Y)Z + XYZ \\&= XY + (X \oplus Y)Z\end{aligned}$$

$$\begin{aligned}S &= X'Y'Z + X'YZ' + XY'Z' + XYZ \\&= X'(Y'Z + YZ') + X(Y'Z' + YZ) \\&= X'(Y \oplus Z) + X(Y \oplus Z)' \\&= X \oplus (Y \oplus Z) \quad \text{or} \quad (X \oplus Y) \oplus Z\end{aligned}$$

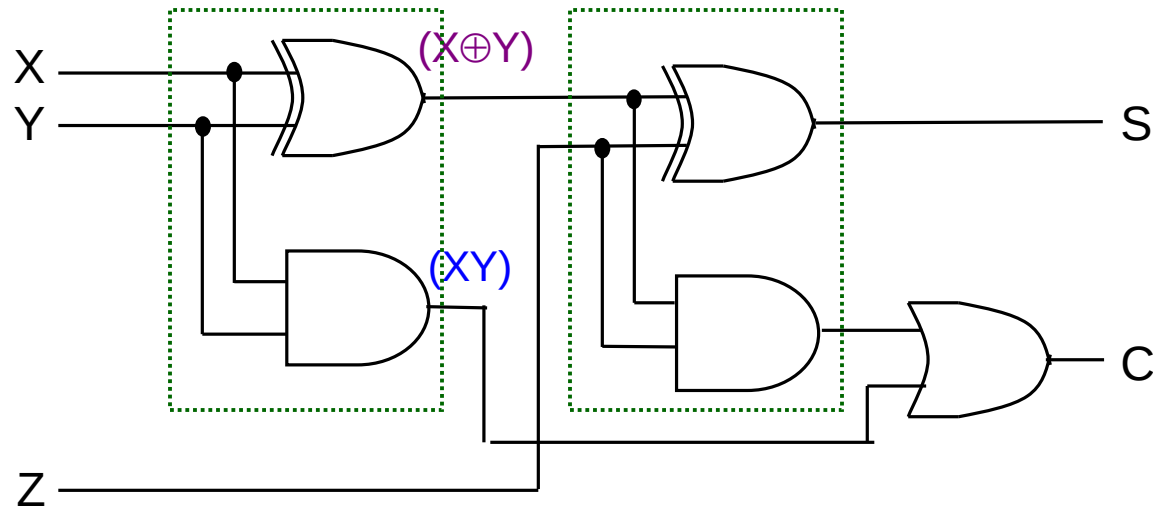


Gate-level (SSI) Design: Full Adder

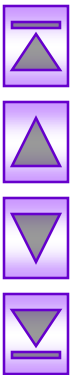
- Circuit for above formulae:

$$C = XY + (X \oplus Y)Z$$

$$S = (X \oplus Y) \oplus Z$$



Full Adder made from two Half-Adders (+ OR gate).

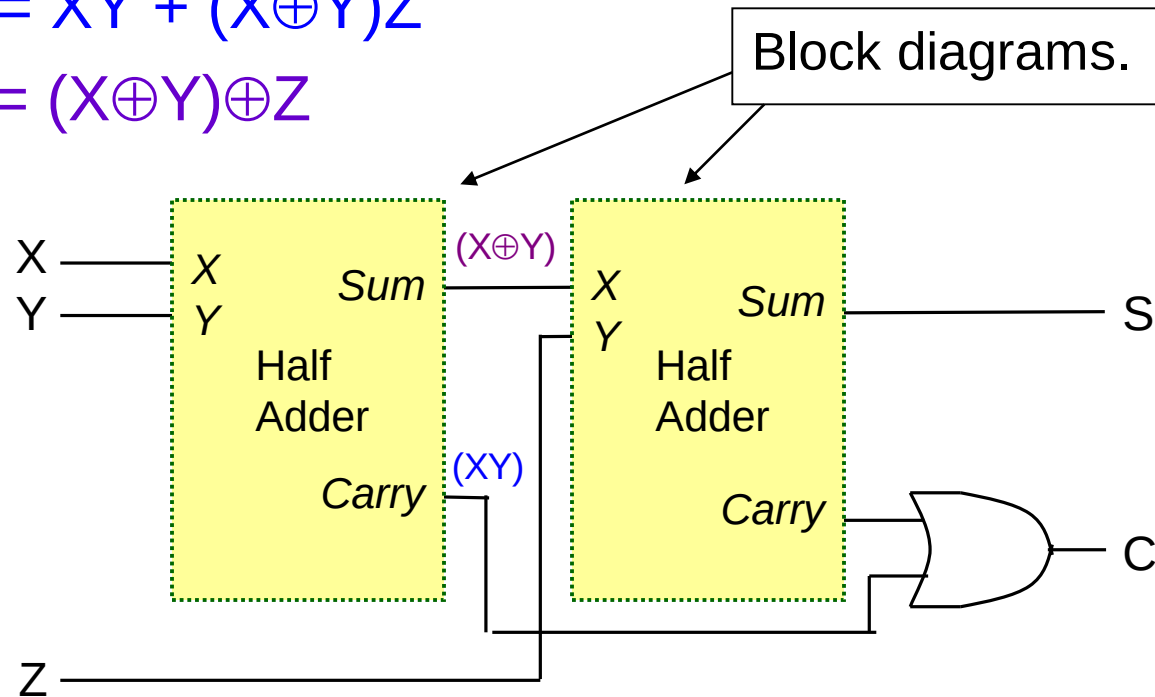


Gate-level (SSI) Design: Full Adder

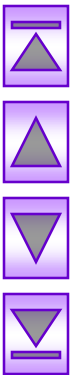
- Circuit for above formulae:

$$C = XY + (X \oplus Y)Z$$

$$S = (X \oplus Y) \oplus Z$$

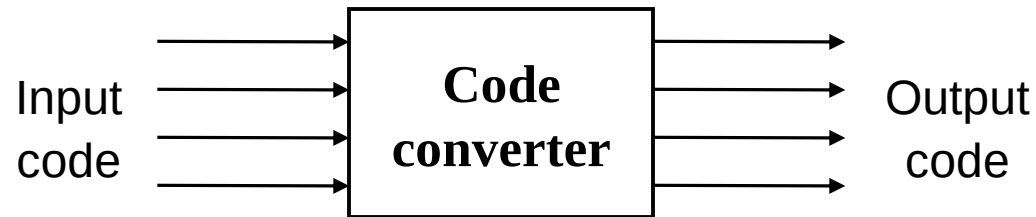


Full Adder made from two Half-Adders (+ OR gate).



Code Converters

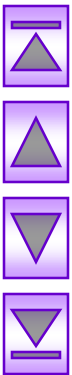
- Code converters – take an input code, translate to its equivalent output code.



- Example: **BCD to Excess-3 Code Converter.**

Input: BCD digit

Output: Excess-3 digit



BCD-to-Excess-3 Code Converter

■ Truth table:

	BCD				Excess-3			
	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0
10	1	0	1	0	X	X	X	X
11	1	0	1	1	X	X	X	X
12	1	1	0	0	X	X	X	X
13	1	1	0	1	X	X	X	X
14	1	1	1	0	X	X	X	X
15	1	1	1	1	X	X	X	X

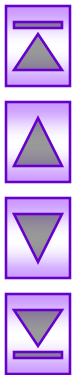
■ K-maps:

AB \ CD		C			
		00	01	11	10
A	00				
	01		1	1	1
	11	X	X	X	X
	10	1	1	X	X
		D			
		W			

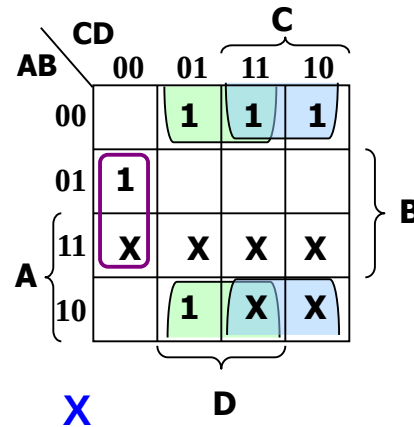
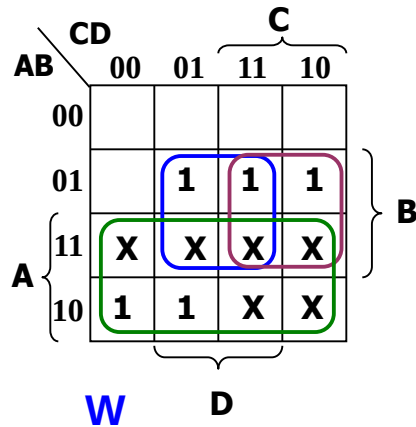
AB \ CD		C			
		00	01	11	10
A	00		1	1	1
	01	1			
	11	X	X	X	X
	10		1	X	X
		D			
		X			

AB \ CD		C			
		00	01	11	10
A	00	1		1	
	01	1		1	
	11	X	X	X	X
	10	1		X	X
		D			
		Y			

AB \ CD		C			
		00	01	11	10
A	00	1			1
	01	1			1
	11	X	X	X	X
	10	1		X	X
		D			
		Z			



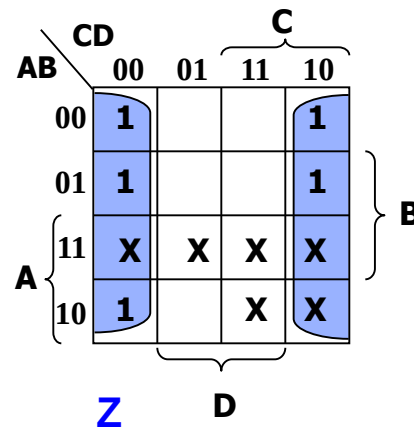
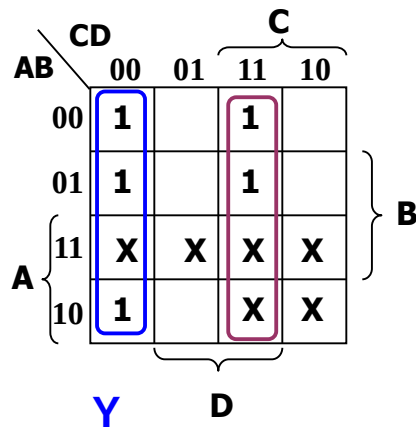
BCD-to-Excess-3 Code Converter



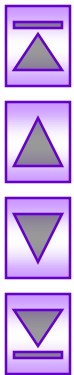
$$W = A + BC + BD$$

$$X = B'C + B'D + BC'D'$$

$$Y = CD + C'D'$$



$$Z = D'$$



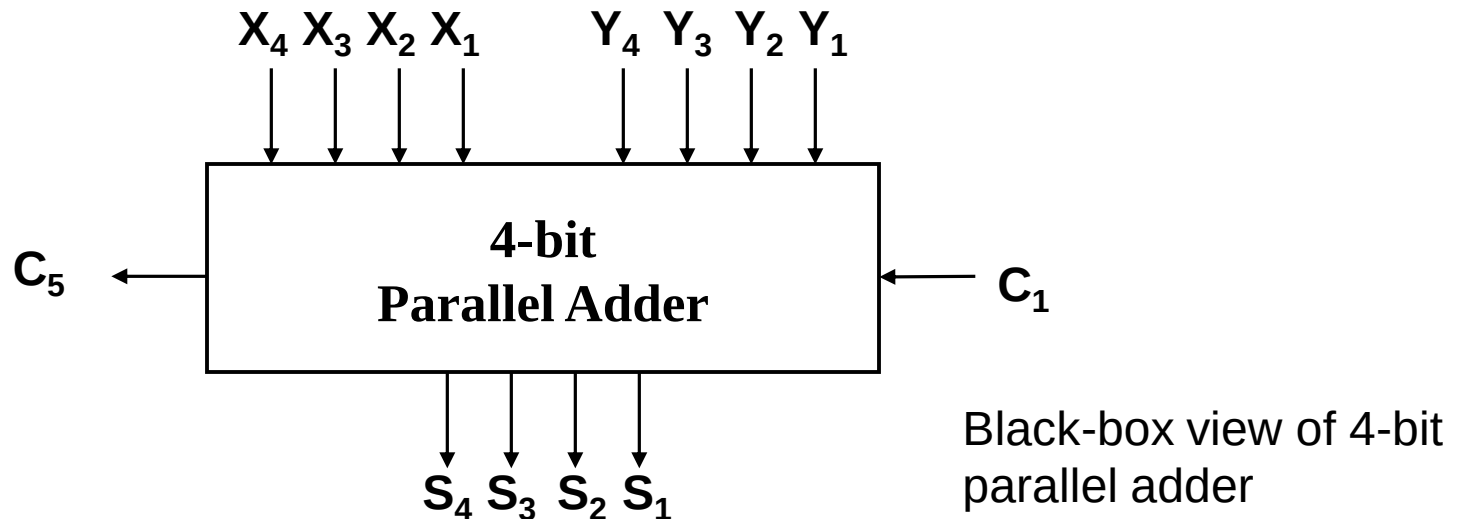
Block-Level Design Method

- More complex circuits can also be built using block-level method.
- In general, block-level design method (as opposed to gate-level design) relies on algorithms or formulae of the circuit, which are obtained by decomposing the main problem to sub-problems recursively (until small enough to be directly solved by blocks of circuits).
- Simple examples using 4-bit parallel adder as building blocks:
 - ❖ (1) BCD-to-Excess-3 Code Conversion
 - ❖ (2) 16-Bit Parallel Adder
 - ❖ (3) Adder cum Subtractor



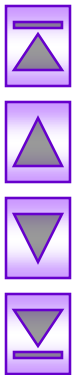
4-bit Parallel Adder

- Consider a circuit to add two 4-bit numbers together and a carry-in, to produce a 5-bit result:



- 5-bit result is sufficient because the largest result is:

$$(1111)_2 + (1111)_2 + (1)_2 = (11111)_2$$

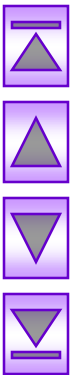


4-bit Parallel Adder

- SSI design technique should not be used.
- Truth table for 9 inputs very big, i.e. $2^9=512$ entries:

$X_4X_3X_2X_1$	$Y_4Y_3Y_2Y_1$	C_1	C_5	$S_4S_3S_2S_1$
0 0 0 0	0 0 0 0	0	0	0 0 0 0
0 0 0 0	0 0 0 0	1	0	0 0 0 1
0 0 0 0	0 0 0 1	0	0	0 0 0 1
...	...	...	...	...
0 1 0 1	1 1 0 1	1	1	0 0 1 1
...	...	...	...	...
1 1 1 1	1 1 1 1	1	1	1 1 1 1

- Simplification very complicated.



4-bit Parallel Adder

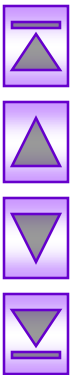
- Alternative design possible.
- Addition formulae for each pair of bits (with carry in),

$$C_{i+1}S_i = X_i + Y_i + C_i$$

has the same function as a full adder.

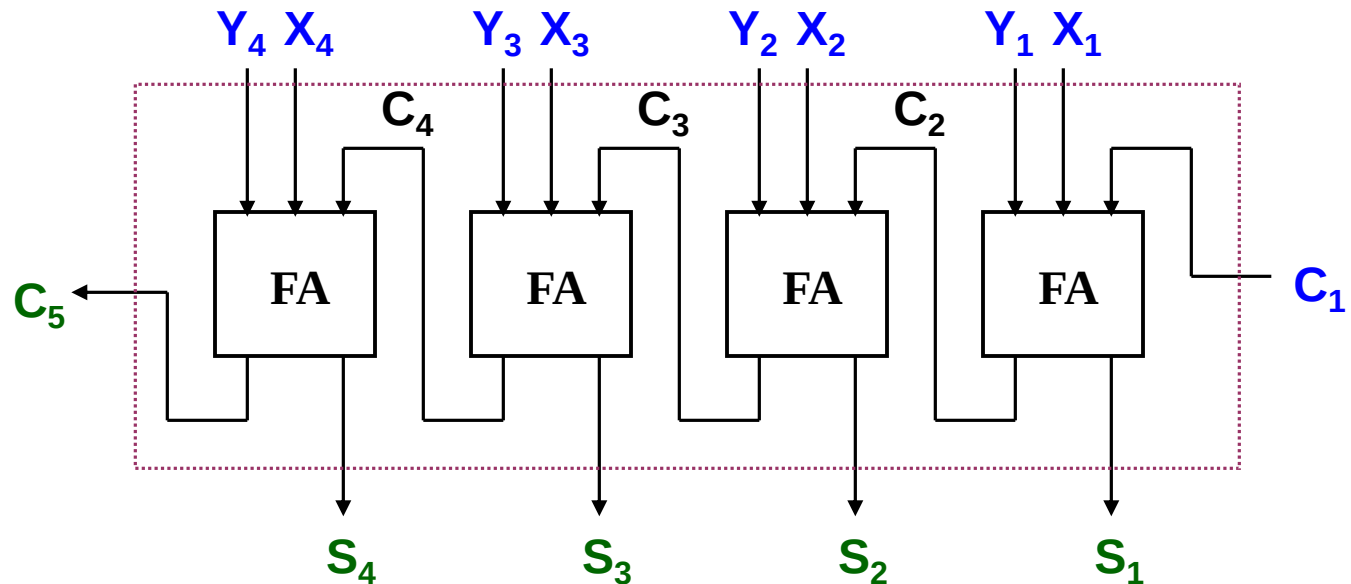
$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$

$$S_i = X_i \oplus Y_i \oplus C_i$$

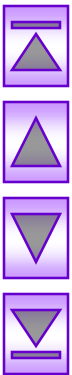


4-bit Parallel Adder

- Cascading 4 full adders via their carries, we get:

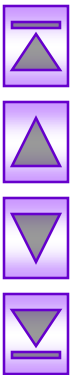


■ Input
■ Output



Parallel Adders

- Note that carry propagated by cascading the carry from one full adder to the next.
- Called **Parallel Adder** because inputs are presented simultaneously (in parallel). Also, called **Ripple-Carry Adder**.



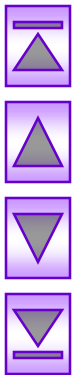
BCD-to-Excess-3 Code Converter

- Excess-3 code can be converted from BCD code using truth table:
- Gate-level design can be used since only 4 inputs.
- However, alternative design possible.
- Use problem-specific formulae:

Excess-3 Code

$$= \text{BCD Code} + (0011)_2$$

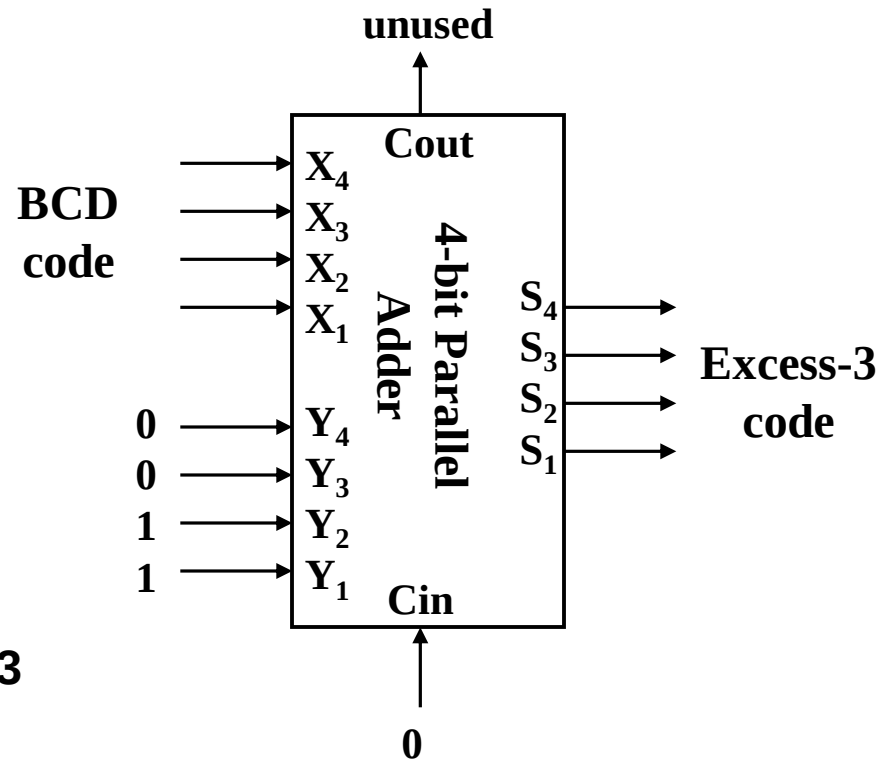
	BCD				Excess-3			
	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0
10	1	0	1	0	X	X	X	X
11	1	0	1	1	X	X	X	X
12	1	1	0	0	X	X	X	X
13	1	1	0	1	X	X	X	X
14	1	1	1	0	X	X	X	X
15	1	1	1	1	X	X	X	X



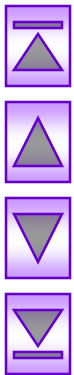
BCD-to-Excess-3 Code Converter

$$\text{Excess-3 Code} = \text{BCD Code} + (0011)_2$$

- Block-level circuit:

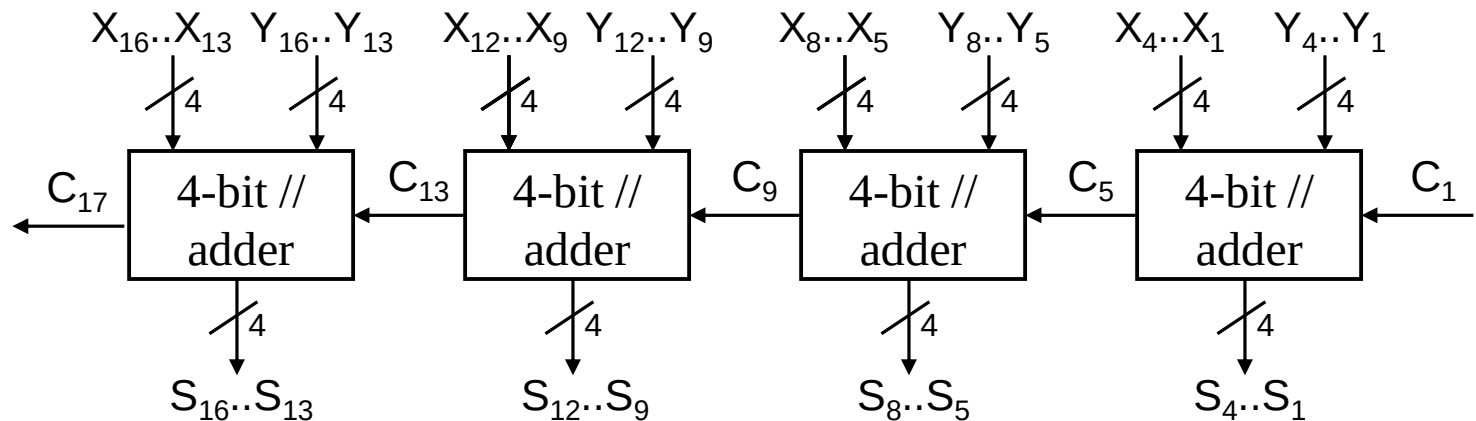


A BCD-to-Excess-3
Code Converter

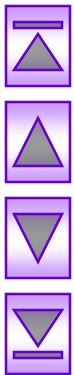


16-bit Parallel Adder

- Larger parallel adders can be built from smaller ones.
- Example: a **16-bit parallel adder** can be constructed from four 4-bit parallel adders:

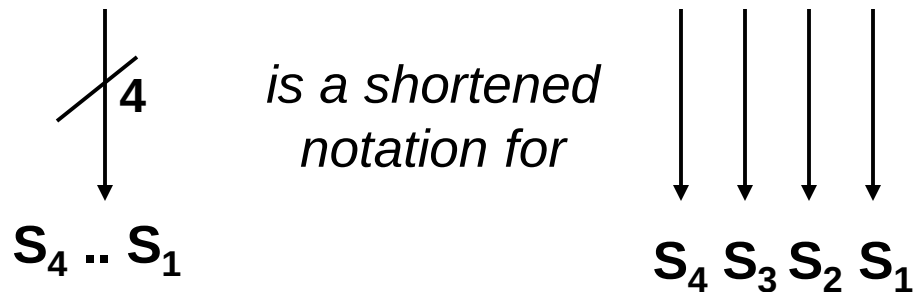


A 16-bit parallel adder



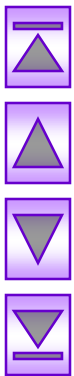
16-bit Parallel Adder

- Shortened notation for multiple lines.



16-bit parallel adder ripples carry from one 4-bit block to the next.

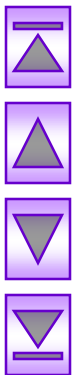
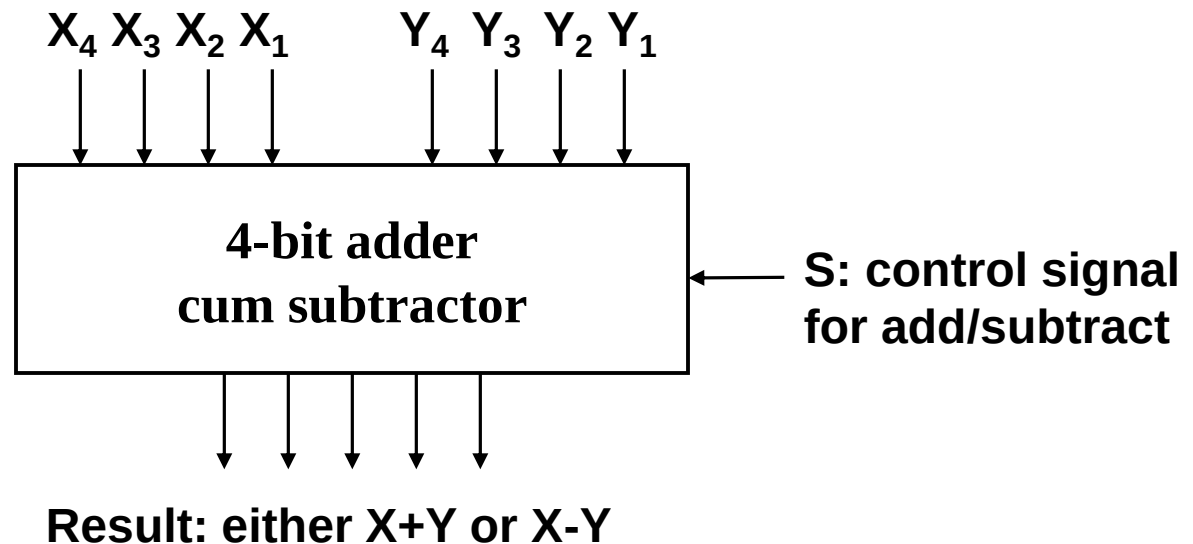
Such ripple-carry circuits are “slow” because of long delays needed to propagate the carries.



4-bit Parallel Adder cum Subtractor

**Optional*

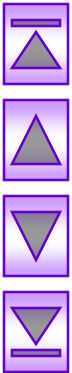
- Subtraction can be performed through addition using 2s-complement numbers.
- Hence, we can design a circuit which can perform **both addition and subtraction**, using a parallel adder.



4-bit Parallel Adder cum Subtractor

- The control signal **S=0** means add
S=1 means subtract
- Recall that:

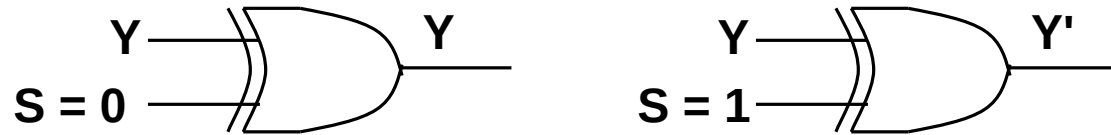
$$\begin{aligned}X - Y &= X + (-Y) \\&= X + (\text{2's complement of } Y) \\&= X + \underline{(\text{1's complement of } Y) + 1} \\X + Y &= X + \underline{(Y)}\end{aligned}$$



4-bit Parallel Adder cum Subtractor

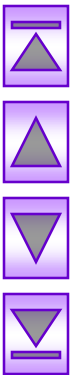
- Design requires:

(i) **XOR gates:**



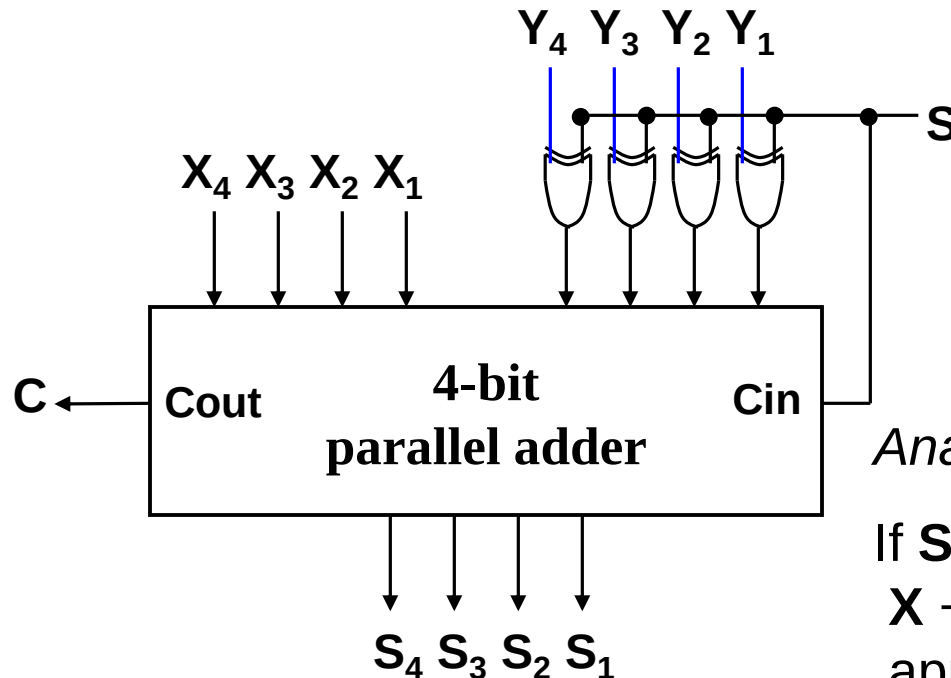
such that: output = Y when $S=0$
 = Y' when $S=1$

(ii) **S connected to carry-in.**



4-bit Parallel Adder cum Subtractor

- Adder cum subtractor circuit:

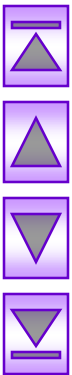


Analysis:

If **S**=1, then
X + (1's complement of **Y**) +1
appears as the result.

If **S**=0, then **X**+**Y** appears as
the result.

A 4-bit adder cum subtractor

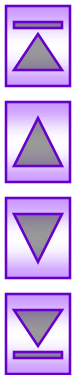
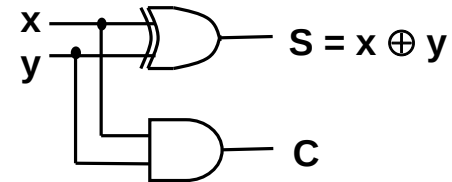
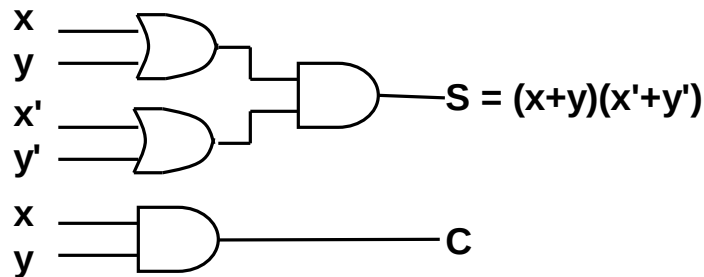
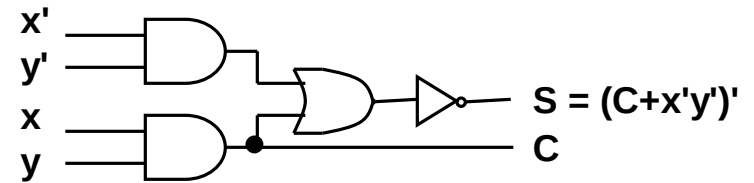
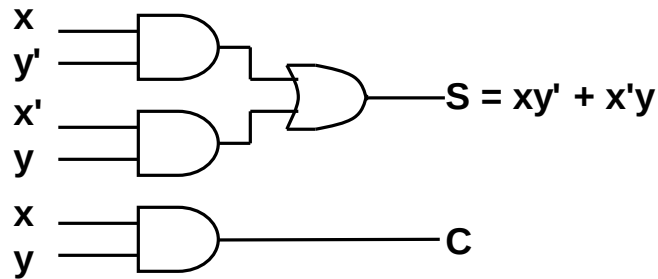
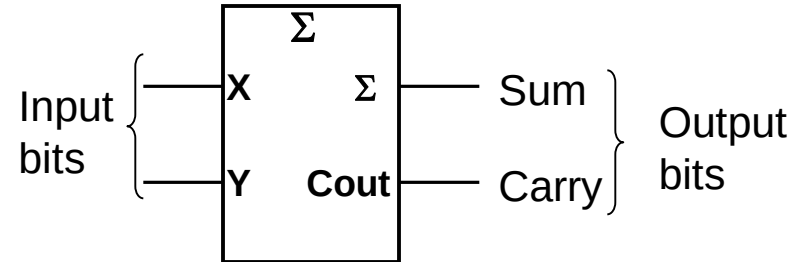


Arithmetic Circuits: Adders

Revision

■ Half adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

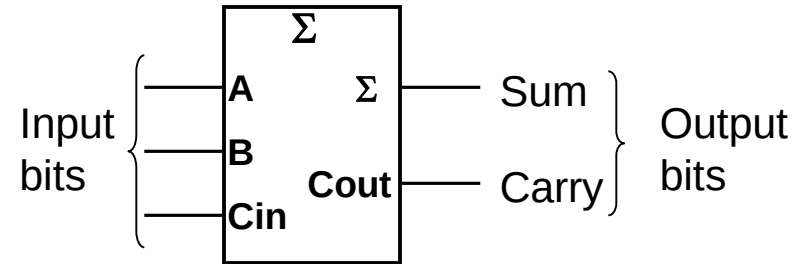
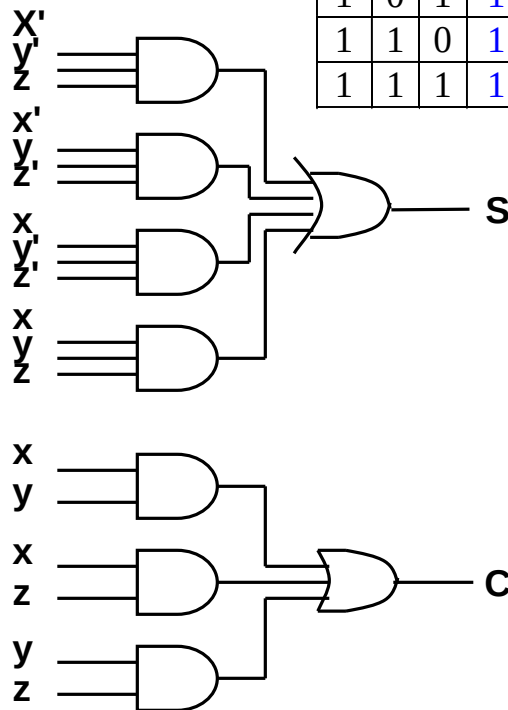


Arithmetic Circuits: Adders

Revision

Full adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

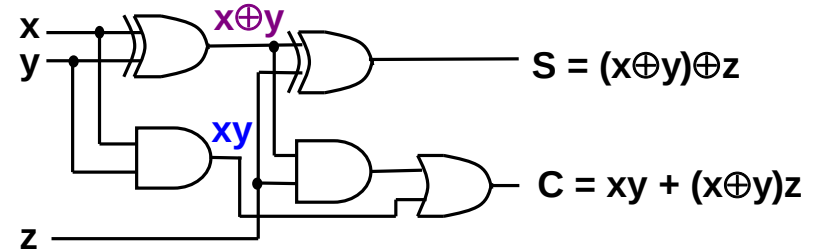


yz	00	01	11	10
x			1	
0			1	
1	1	1	1	

$C = xy + xz + yz$

yz	00	01	11	10
x		1		1
0		1		1
1	1		1	

$S = x'y'z + x'yz' + xy'z' + xyz$



Arithmetic Circuits: Parallel Adders

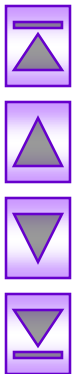
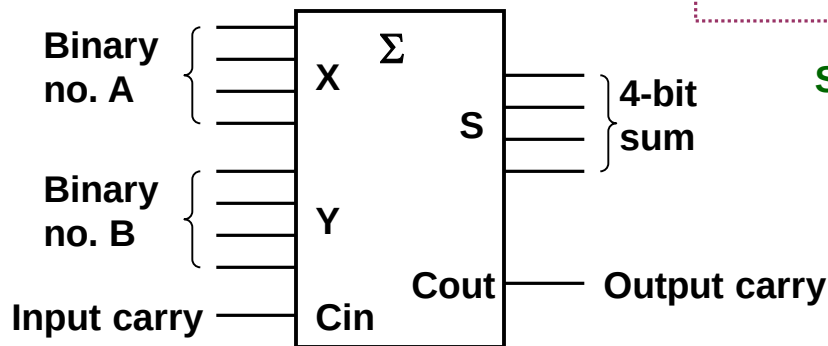
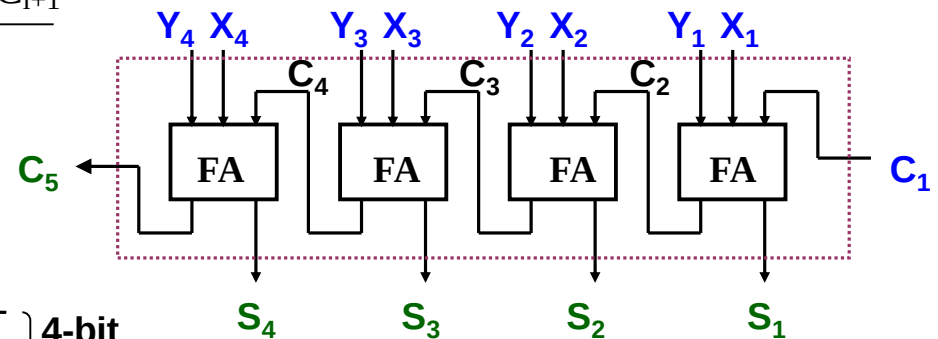
Revision

- Example: Adding two 4-bit numbers

Subscript i	4	3	2	1	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

2 ways:

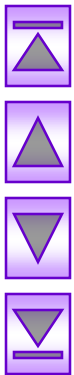
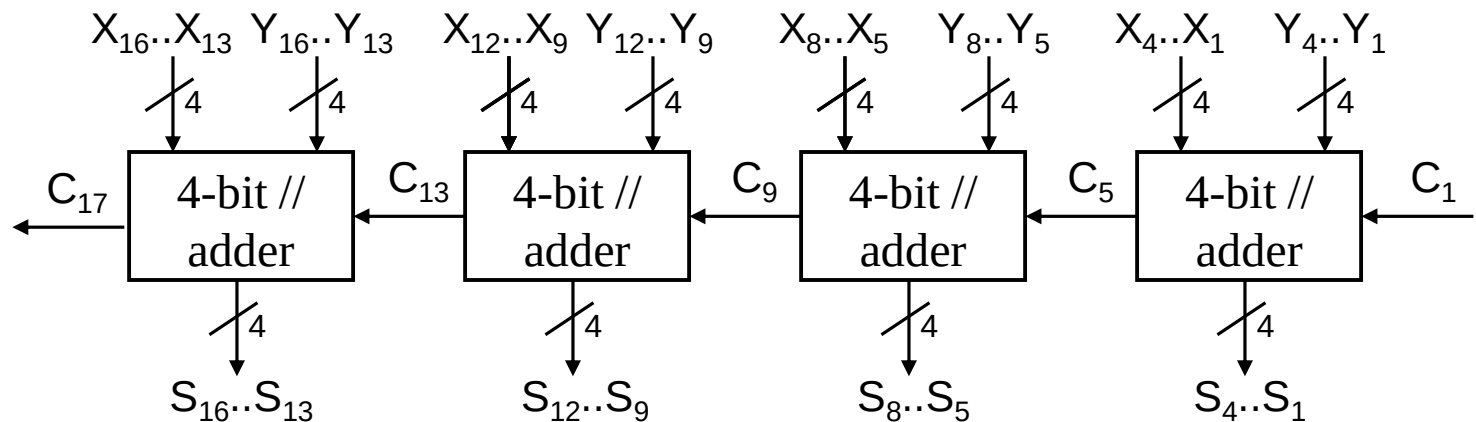
- Serial (one FA)
- Parallel (n FAs for n bits)



Arithmetic Circuits: Cascading Adders

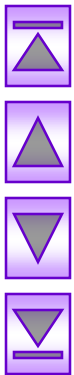
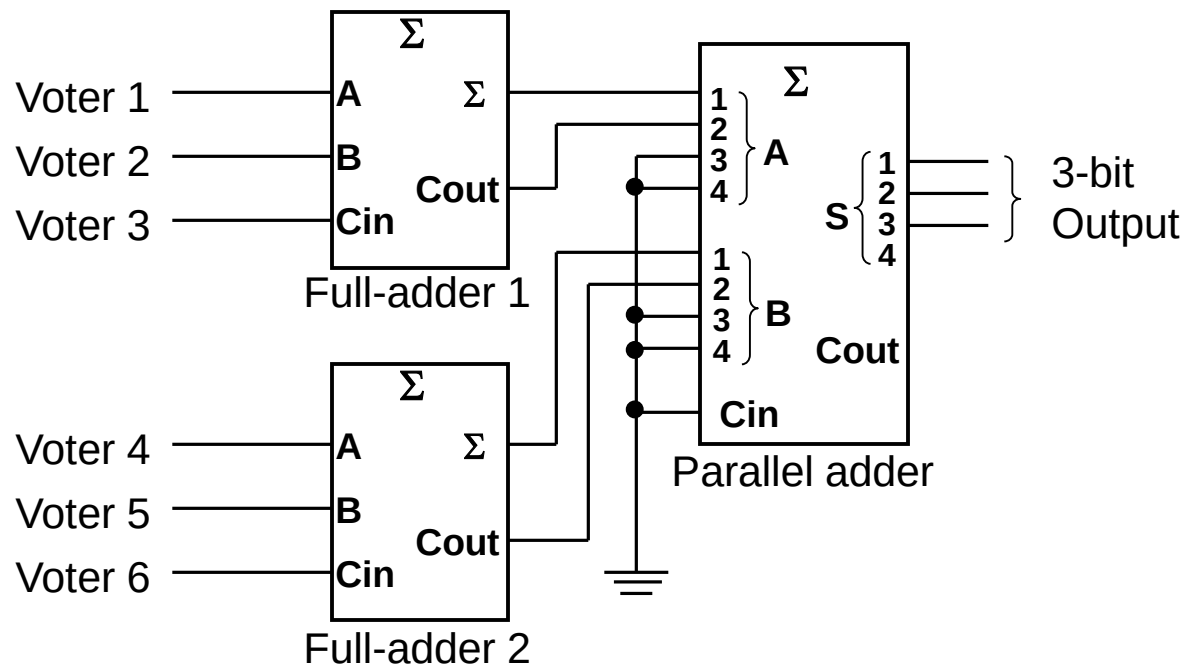
Revision

- 4-bit parallel adder:
 - ❖ cascade 4 full adders
 - ❖ classical method: 9 input variables $\bowtie 2^9 = 512$ rows in truth table!
- Cascading method can be extended to larger numbers, example: 16-bit parallel adder.



Arithmetic Circuits: Cascading Adders

- Application: 6-person voting system.
 - ❖ Use FAs and a 4-bit binary parallel adder.
 - ❖ Each FA can sum up to 3 votes.



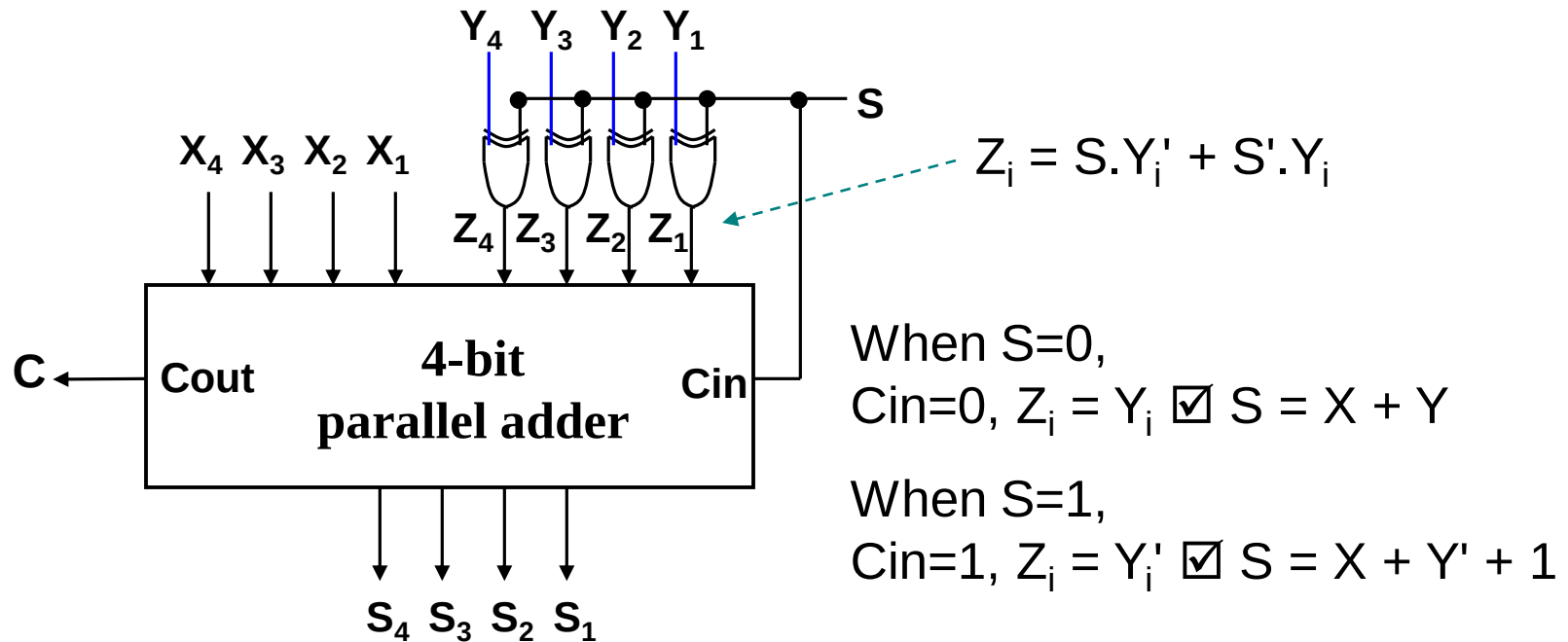
Arithmetic Circuits: Adder-Subtractor

Revision

- Make use of 2's complement:

$$X - Y = X + (-Y)$$

- 2's complement of Y = Inverting bits in Y and plus 1.



Arithmetic Circuits: Comparator

- **Magnitude comparator**: compares 2 values A and B, to see if $A > B$, $A = B$ or $A < B$.
- Classical method requires 2^{2n} rows in truth table!
- We exploit regularity.
- How do we compare two 4-bit values A ($a_3a_2a_1a_0$) and B ($b_3b_2b_1b_0$)?



If ($a_3 > b_3$) then $A > B$



If ($a_3 < b_3$) then $A < B$

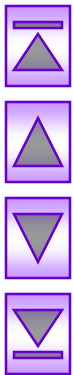
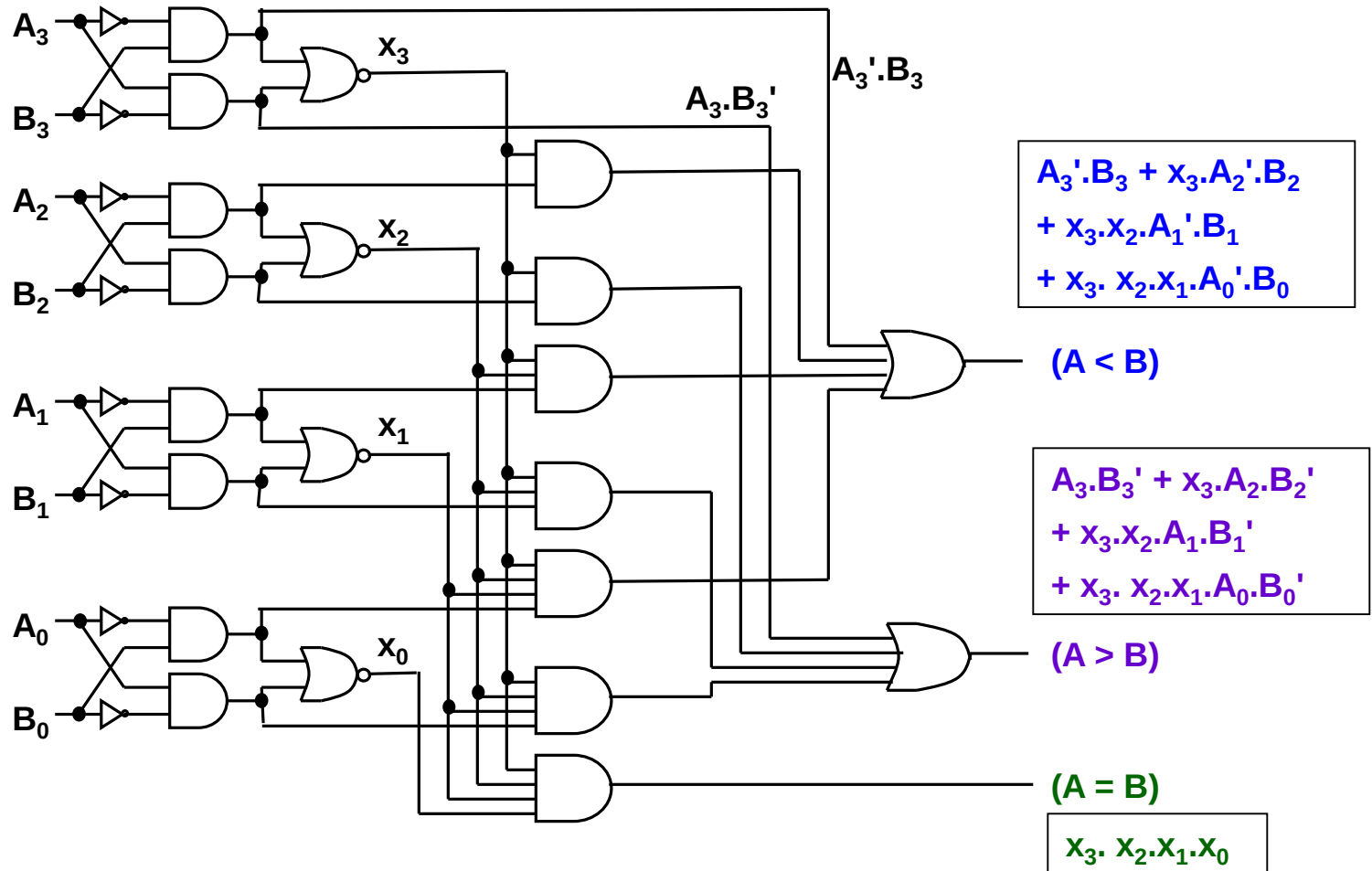


If ($a_3 = b_3$) then if ($a_2 > b_2$)

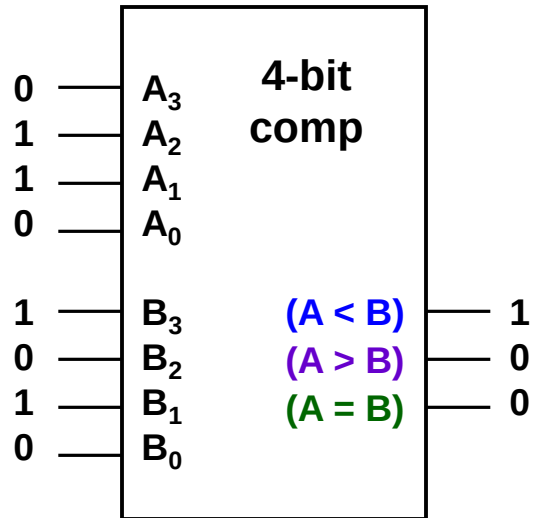


Arithmetic Circuits: Comparator

Let $A = A_3A_2A_1A_0$, $B = B_3B_2B_1B_0$; $x_i = A_i \cdot B_i + A_i' \cdot B_i'$



Arithmetic Circuits: Comparator



Block diagram of a 4-bit magnitude comparator

